**Group A & B**

**Design Decisions**

- Option Class (Abstract Base)
    - The Option class is designed as an abstract parent class for all option types, with shared state variables (K, \sigma, r, S, b) and a pure virtual pricing interface. This ensures code reuse and enforces a consistent contract for derived classes (each must implement its own pricing logic).
    - The OptionType enum (Call/Put) improves readability and type safety compared with string-based option flags.
    - Two overloaded constructors are provided. The constructor without (b) is convenient for standard stock-option settings where (b=r), while the constructor with (b) supports generalized cost-of-carry models (dividend yield, futures, FX) where (b \neq r). This gives both simplicity for common cases and flexibility for advanced cases.
    - validateCommonInputs(S, K, sigma): I centralized basic parameter checks in the base class so all derived options inherit the same safety rules. This avoids duplicated validation logic and inconsistent error behavior.
    - price() as pure virtual: I made pricing abstract so each option type must provide its own model-specific formula. This enforces clean polymorphism and keeps model differences explicit.
- European option class
    - EuropeanOption derives from Option and implements closed-form BS pricing. It includes internal helper functions (d_1) and (d_2), plus analytic Greeks (Delta and Gamma), so pricing and sensitivity analysis are both available from the same object. Input checks (e.g., (T>0)) are included to prevent invalid model states.
    - EuropeanOption constructors: I mirrored the base-class constructor overloads to keep object creation consistent across classes. This design makes it easy to switch between default and generalized carry assumptions.
    - validateMaturity(T): I validate maturity at construction time to fail fast on invalid contracts. That prevents downstream formula errors and keeps diagnostics clear.
    - d1() and d2(): I isolated BS intermediates into helper functions to reduce repetition in price and Greek calculations. This improves readability and lowers the chance of formula mistakes.
    - price(): I implemented closed-form BS pricing to provide exact benchmark values for testing. It is the core valuation function for European call/put contracts.
    - delta(): Function to calculate delta. Keeping it in-class ensures consistency with the same model assumptions as pricing.
    - gamma(): Function to calculate gamma. Putting it in the same class keeps all core Greeks cohesive.
- American option class

    - PerpetualAmericanOption derives from Option and implements the closed-form perpetual American call/put formulas. Since maturity is infinite, this class has a different model structure from European options and is separated to keep responsibilities clear. It also includes numerical safeguards near singular parameter regions to avoid unstable computations.
    - PerpetualAmericanOption constructors: I kept constructor consistent with other option classes for a uniform object creation. This reduces cognitive load when switching models in main/testing code.
    - price(): I override the price function to implement the closed-form solution for perpetual American options. I also added numerical guards to avoid unstable divisions near singular parameter regions.
- Utility files:
    - Utility logic is separated from pricing classes into three headers: CommonUtilities, EuropeanUtilities, and PerpetualAmericanUtilities. CommonUtilities contains reusable infrastructure (input structures, mesh generation, matrix evaluation, and formatted output), while each model-specific utility file contains domain-specific helpers.
    - Common Utilities:
        - MeshArray(start, end, h): I created this to standardize mesh generation for asset and parameter sweeps. A single helper makes experiments repeatable and avoids loop code everywhere.
        - formatParameterValue(value): I use one formatter so matrix labels are consistent, and the output is cleaner. This improves readability when comparing many scenarios.
        - withParameterValue(input, parameter, value): Function to create a new OptionInput with one parameter value changed, used for generating input matrices by varying a specific parameter.
        - buildInputMatrixFromSweep(...): I added this to convert sweep vectors into structured input matrices for batch evaluation. It separates data generation from pricing logic.
        - evaluateMatrix(matrix, evaluator): I designed this as a generic execution engine so pricing/Greeks can be swapped via functors without changing matrix traversal code. This makes the framework reusable and extensible.
        - printLine(), printSeries(...), printMatrix(...): I included these to keep output formatting consistent across all tasks.
    - EuropeanUtilities
        - europeanBatches(): I created fixed benchmark batches so results are repeatable and easy to compare against expected values.
        - parityPutFromCall(...) and parityCallFromPut(...): I included both conversion directions to verify pricing consistency from either known value.
        - satisfiesParity(...): I added a parity check to indicate a pass/fail signal for validation.

        - europeanPriceAcrossS(...): Function to calculate European option prices across a range of asset prices
        - europeanDeltaAcrossS(...) and europeanGammaAcrossS(...): Function to calculate the delta and gamma of European options across a range of asset prices
        - finiteDifferenceDelta(...) and finiteDifferenceGamma(...): Function to calculate the delta and gamma of a European option using finite difference approximation
    - PerpetualAmericanUtilities:
        - perpetualPrice(...): I added a simple wrapper for one-point pricing so main code stays concise and model calls remain explicit. It improves readability in batch/test sections.
        - perpetualPriceAcrossS(...): Function to calculate perpetual American option prices across a range of asset prices
- Main: Test Program
    - The main program acts as an integration and validation driver for all tasks in the outline. It executes batch pricing, put-call parity checks, mesh-based sweeps, Greek calculations, and finite-difference comparisons. It also runs the perpetual American test cases and matrix evaluations.

**A: Exact Solutions of One-Factor Plain Options**

a) Call and put option pricing using the data sets Batch 1 to Batch 4.
    - Batch 1 C = 2.133368 P = 5.846282
    - Batch 2 C = 7.965567 P = 7.965567
    - Batch 3 C = 0.204058 P = 4.073262
    - Batch 4 C = 92.175704 P = 1.247499

b) Apply the put-call parity relationship to compute call and put option prices.
    - Batch 1 PutFromCall = 5.846282 CallFromPut = 2.133368 Parity = Satisfied
    - Batch 2 PutFromCall = 7.965567 CallFromPut = 7.965567 Parity = Satisfied
    - Batch 3 PutFromCall = 4.073262 CallFromPut = 0.204058 Parity = Satisfied
    - Batch 4 PutFromCall = 1.247499 CallFromPut = 92.175704 Parity = Satisfied

c) Compute option prices for a monotonically increasing range of underlying values of S, for example 10, 11, 12, …, 50
    - Call prices on S mesh :
    - S=10.000000 --> 0.000000
    - S=11.000000 --> 0.000000
    - S=12.000000 --> 0.000000
    - S=13.000000 --> 0.000000
    - S=14.000000 --> 0.000000
    - S=15.000000 --> 0.000000
    - S=16.000000 --> 0.000000
    - S=17.000000 --> 0.000000
    - S=18.000000 --> 0.000000
    - S=19.000000 --> 0.000000

- S=20.000000 --> 0.000000
- S=21.000000 --> 0.000000
- S=22.000000 --> 0.000000
- S=23.000000 --> 0.000000
- S=24.000000 --> 0.000000
- S=25.000000 --> 0.000000
- S=26.000000 --> 0.000000
- S=27.000000 --> 0.000000
- S=28.000000 --> 0.000000
- S=29.000000 --> 0.000000
- S=30.000000 --> 0.000000
- S=31.000000 --> 0.000001
- S=32.000000 --> 0.000003
- S=33.000000 --> 0.000008
- S=34.000000 --> 0.000021
- S=35.000000 --> 0.000052
- S=36.000000 --> 0.000119
- S=37.000000 --> 0.000259
- S=38.000000 --> 0.000534
- S=39.000000 --> 0.001055
- S=40.000000 --> 0.001994
- S=41.000000 --> 0.003622
- S=42.000000 --> 0.006339
- S=43.000000 --> 0.010714
- S=44.000000 --> 0.017528
- S=45.000000 --> 0.027817
- S=46.000000 --> 0.042908
- S=47.000000 --> 0.064447
- S=48.000000 --> 0.094413
- S=49.000000 --> 0.135117
- S=50.000000 --> 0.189181
- Put prices on S mesh :
- S=10.000000 --> 53.712914
- S=11.000000 --> 52.712914
- S=12.000000 --> 51.712914
- S=13.000000 --> 50.712914
- S=14.000000 --> 49.712914
- S=15.000000 --> 48.712914
- S=16.000000 --> 47.712914
- S=17.000000 --> 46.712914
- S=18.000000 --> 45.712914
- S=19.000000 --> 44.712914
- S=20.000000 --> 43.712914
- S=21.000000 --> 42.712914

- S=22.000000 --> 41.712914
- S=23.000000 --> 40.712914
- S=24.000000 --> 39.712914
- S=25.000000 --> 38.712914
- S=26.000000 --> 37.712914
- S=27.000000 --> 36.712914
- S=28.000000 --> 35.712914
- S=29.000000 --> 34.712914
- S=30.000000 --> 33.712914
- S=31.000000 --> 32.712915
- S=32.000000 --> 31.712917
- S=33.000000 --> 30.712922
- S=34.000000 --> 29.712935
- S=35.000000 --> 28.712966
- S=36.000000 --> 27.713033
- S=37.000000 --> 26.713172
- S=38.000000 --> 25.713448
- S=39.000000 --> 24.713969
- S=40.000000 --> 23.714908
- S=41.000000 --> 22.716536
- S=42.000000 --> 21.719253
- S=43.000000 --> 20.723628
- S=44.000000 --> 19.730442
- S=45.000000 --> 18.740731
- S=46.000000 --> 17.755822
- S=47.000000 --> 16.777361
- S=48.000000 --> 15.807326
- S=49.000000 --> 14.848031
- S=50.000000 --> 13.902095

d) Extend part c and compute option prices as a function of i) expiry time, ii) volatility, or iii) any of the option pricing parameters.
   - European call price matrix (T sweep):
   - Row 1: 2.133368
   - Row 2: 5.675400
   - Row 3: 8.518788
   - Row 4: 11.011762
   - Row 5: 13.272445
   - European call price matrix (sigma sweep):
   - Row 1: 0.173097
   - Row 2: 1.041454
   - Row 3: 2.133368
   - Row 4: 3.290333
   - Row 5: 4.472438
   - European call price matrix (r sweep):

- Row 1: 2.176465
- Row 2: 2.160203
- Row 3: 2.144062
- Row 4: 2.128042
- Row 5: 2.112141

**Option Sensitivities, aka the Greeks**

a) Implement the above formulae for gamma for call and put future option pricing using the data set: K = 100, S = 105, T = 0.5, r = 0.1, b = 0 and sig = 0.36. (exact delta call = 0.5946, delta put = -0.3566)
   - Exact call delta=0.594629 (target 0.5946)
   - Exact put delta =-0.356601 (target -0.3566)
   - Exact gamma =0.013494 (same for call/put)

b) Compute call delta price for a monotonically increasing range of underlying values of S, for example 10, 11, 12, …, 50.
   - Call delta on S mesh:
   - S=10.000000 --> 0.000000
   - S=11.000000 --> 0.000000
   - S=12.000000 --> 0.000000
   - S=13.000000 --> 0.000000
   - S=14.000000 --> 0.000000
   - S=15.000000 --> 0.000000
   - S=16.000000 --> 0.000000
   - S=17.000000 --> 0.000000
   - S=18.000000 --> 0.000000
   - S=19.000000 --> 0.000000
   - S=20.000000 --> 0.000000
   - S=21.000000 --> 0.000000
   - S=22.000000 --> 0.000000
   - S=23.000000 --> 0.000000
   - S=24.000000 --> 0.000000
   - S=25.000000 --> 0.000000
   - S=26.000000 --> 0.000000
   - S=27.000000 --> 0.000000
   - S=28.000000 --> 0.000001
   - S=29.000000 --> 0.000001
   - S=30.000000 --> 0.000002
   - S=31.000000 --> 0.000004
   - S=32.000000 --> 0.000007
   - S=33.000000 --> 0.000011
   - S=34.000000 --> 0.000019
   - S=35.000000 --> 0.000031
   - S=36.000000 --> 0.000048
   - S=37.000000 --> 0.000075

- S=38.000000 --> 0.000114
- S=39.000000 --> 0.000169
- S=40.000000 --> 0.000245
- S=41.000000 --> 0.000351
- S=42.000000 --> 0.000493
- S=43.000000 --> 0.000681
- S=44.000000 --> 0.000927
- S=45.000000 --> 0.001244
- S=46.000000 --> 0.001648
- S=47.000000 --> 0.002154
- S=48.000000 --> 0.002783
- S=49.000000 --> 0.003554
- S=50.000000 --> 0.004490

c) Incorporate this into your above matrix pricer code, so you can input a matrix of option parameters and receive a matrix of either Delta or Gamma as the result.
   - Delta matrix:
   - Row 1: 0.119827
   - Row 2: 0.291013
   - Row 3: 0.372483
   - Row 4: 0.420657
   - Row 5: 0.454155
   - Gamma matrix:
   - Row 1: 0.066612
   - Row 2: 0.057144
   - Row 3: 0.042043
   - Row 4: 0.032585
   - Row 5: 0.026420

d) perform the same calculations as in parts a and b, but now using divided differences. Compare the accuracy with various values of the parameter h
   - h=0.100000 | FD call delta = 0.594628 | |delta error| = 0.000000 | FD call gamma = 0.013494 | |gamma error| = 0.000000
   - h=5.100000 | FD call delta = 0.593381 | |delta error| = 0.001247 | FD call gamma = 0.013472 | |gamma error| = 0.000022
   - h=10.100000 | FD call delta = 0.589826 | |delta error| = 0.004803 | FD call gamma = 0.013408 | |gamma error| = 0.000086
   - h=15.100000 | FD call delta = 0.584216 | |delta error| = 0.010413 | FD call gamma = 0.013299 | |gamma error| = 0.000194
   - h=20.100000 | FD call delta = 0.576951 | |delta error| = 0.017678 | FD call gamma = 0.013143 | |gamma error| = 0.000351
   - h=25.100000 | FD call delta = 0.568540 | |delta error| = 0.026089 | FD call gamma = 0.012936 | |gamma error| = 0.000558
   - h=30.100000 | FD call delta = 0.559550 | |delta error| = 0.035079 | FD call gamma = 0.012677 | |gamma error| = 0.000817

- h=35.100000 | FD call delta = 0.550546 | |delta error| = 0.044083 | FD call gamma = 0.012368 | |gamma error| = 0.001126
- h=40.100000 | FD call delta = 0.542019 | |delta error| = 0.052609 | FD call gamma = 0.012013 | |gamma error| = 0.001481
- h=45.100000 | FD call delta = 0.534333 | |delta error| = 0.060296 | FD call gamma = 0.011621 | |gamma error| = 0.001873
- h=50.100000 | FD call delta = 0.527685 | |delta error| = 0.066944 | FD call gamma = 0.011203 | |gamma error| = 0.002291

**B: Perpetual American Options**

a) Program the above formulae, and incorporate into your well-designed options pricing classes
b) Test the data with K = 100, sig = 0.1, r = 0.1, b = 0.02, S = 110 (check C = 18.5035, P = 3.03106)
    - Perpetual test | C=18.503500 (target 18.5035) | P=3.031060 (target 3.03106)
c) Compute call and put option price for a monotonically increasing range of underlying values of S, for example 10, 11, 12, …, 50.
    - Perpetual call prices on S mesh:
    - S=10.000000 --> 0.008262
    - S=11.000000 --> 0.011227
    - S=12.000000 --> 0.014853
    - S=13.000000 --> 0.019216
    - S=14.000000 --> 0.024389
    - S=15.000000 --> 0.030450
    - S=16.000000 --> 0.037476
    - S=17.000000 --> 0.045547
    - S=18.000000 --> 0.054741
    - S=19.000000 --> 0.065141
    - S=20.000000 --> 0.076827
    - S=21.000000 --> 0.089883
    - S=22.000000 --> 0.104394
    - S=23.000000 --> 0.120442
    - S=24.000000 --> 0.138115
    - S=25.000000 --> 0.157497
    - S=26.000000 --> 0.178677
    - S=27.000000 --> 0.201742
    - S=28.000000 --> 0.226781
    - S=29.000000 --> 0.253883
    - S=30.000000 --> 0.283138
    - S=31.000000 --> 0.314637
    - S=32.000000 --> 0.348471
    - S=33.000000 --> 0.384732
    - S=34.000000 --> 0.423512
    - S=35.000000 --> 0.464906

- S=36.000000 -->  0.509007
- S=37.000000 -->  0.555908
- S=38.000000 -->  0.605706
- S=39.000000 -->  0.658495
- S=40.000000 -->  0.714373
- S=41.000000 -->  0.773434
- S=42.000000 -->  0.835777
- S=43.000000 -->  0.901499
- S=44.000000 -->  0.970699
- S=45.000000 -->  1.043475
- S=46.000000 -->  1.119925
- S=47.000000 -->  1.200151
- S=48.000000 -->  1.284251
- S=49.000000 -->  1.372327
- S=50.000000 -->  1.464480
- Perpetual put prices on S mesh:
- S=10.000000 --> 9034894.535861
- S=11.000000 --> 4995571.212619
- S=12.000000 --> 2908399.275132
- S=13.000000 --> 1768228.316333
- S=14.000000 --> 1115440.651890
- S=15.000000 --> 726382.530919
- S=16.000000 --> 486307.627036
- S=17.000000 --> 333598.675180
- S=18.000000 --> 233827.901146
- S=19.000000 --> 167076.196531
- S=20.000000 --> 121456.970490
- S=21.000000 --> 89678.590115
- S=22.000000 --> 67155.952174
- S=23.000000 --> 50940.657177
- S=24.000000 --> 39097.895778
- S=25.000000 --> 30334.288134
- S=26.000000 --> 23770.466117
- S=27.000000 --> 18799.168265
- S=28.000000 --> 14994.977728
- S=29.000000 --> 12055.885003
- S=30.000000 -->  9764.831374
- S=31.000000 -->  7964.015055
- S=32.000000 -->  6537.480971
- S=33.000000 -->  5399.167673
- S=34.000000 -->  4484.599603
- S=35.000000 -->  3745.046276
- S=36.000000 -->  3143.371334
- S=37.000000 -->  2651.052506

- S=38.000000 --> 2246.021643
- S=39.000000 --> 1911.084991
- S=40.000000 --> 1632.757928
- S=41.000000 --> 1400.398482
- S=42.000000 --> 1205.558054
- S=43.000000 --> 1041.491369
- S=44.000000 --> 902.784030
- S=45.000000 --> 785.067595
- S=46.000000 --> 684.800234
- S=47.000000 --> 599.096863
- S=48.000000 --> 525.596835
- S=49.000000 --> 462.360334
- S=50.000000 --> 407.786801

d) Incorporate this into your above matrix pricer code, so you can input a matrix of option parameters and receive a matrix of Perpetual American option prices.
   - Perpetual American call matrix (r sweep):
   - Row 1: 30.250000
   - Row 2: 18.503500
   - Row 3: 14.938510
   - Row 4: 13.193298
   - Row 5: 12.165044
   - Perpetual American put matrix (r sweep):
   - Row 1: 4.158906
   - Row 2: 3.031060
   - Row 3: 2.418572
   - Row 4: 2.020207
   - Row 5: 1.735795